Interactive Plots with ggiraph

2026-04-17

Introduction

Where `plotly` translates `ggplot2` figures into a separate JavaScript engine, `ggiraph` keeps the original `ggplot2` rendering and adds SVG event handlers to individual elements — points, bars, polygons. The result is interactivity that preserves `ggplot2`'s native typography, themes, and palettes, with sharp vector output that scales cleanly to print and screen alike. Tooltips, click events, hover highlighting, and Shiny linkage come from a small set of `*_interactive` geoms that mirror their `ggplot2` counterparts.

Prerequisites

A working knowledge of `ggplot2` and an output environment that supports HTML — Quarto, RMarkdown, Shiny, or any browser-based viewer.

Theory

Each interactive geom (`geom_point_interactive()`, `geom_col_interactive()`, `geom_polygon_interactive()`, etc.) accepts additional aesthetics on top of its `ggplot2` base:

- `tooltip`: text (or HTML) shown on hover.
- `data_id`: identifier used for cross-element highlighting and selection.
- `onclick`: arbitrary JavaScript invoked when the element is clicked.

The `girafe()` function wraps the resulting plot into an HTML widget; `opts_*` helpers tune hover CSS, selection behaviour, zoom, toolbar visibility, and download options. Because the output is plain SVG, every interactive plot remains a standalone vector graphic that can be exported, embedded, or re-styled with external CSS.

Assumptions

The output target supports HTML and SVG rendering. For PDF or PNG output, `ggiraph` falls back to a static SVG snapshot — interactivity is lost but typography is preserved, unlike `plotly`'s rasterised exports.

R Implementation

```
library(ggplot2); library(ggiraph)

p <- ggplot(mtcars, aes(wt, mpg,
                        tooltip = rownames(mtcars),
                        data_id = rownames(mtcars))) +
  geom_point_interactive(size = 3, colour = "#2A9D8F") +
  theme_minimal()

girafe(ggobj = p,
       options = list(opts_hover(css = "fill:#F4A261;stroke:black"),
                      opts_selection(type = "multiple")))
```

Output & Results

An interactive SVG scatterplot in which hovering a point highlights it with the configured CSS rule, clicking selects it, and shift-clicking adds to the selection. Inside Shiny, the selection propagates to the `input$<plot_id>_selected` reactive, enabling brushed cross-filtering between charts and tables.

Interpretation

A reporting sentence: "Figure 2 (rendered with `ggiraph`): each point shows a vehicle; hover reveals the model name and selected points propagate to the linked summary table below." Mention the technology and any selection semantics in the caption — readers should know whether interactivity is decorative or part of the analytical workflow.

Practical Tips

- Each `geom_*_interactive()` accepts the same arguments as its non-interactive parent plus the new aesthetics, so refactoring an existing plot is a one-word swap per layer.
- Use `data_id` to group elements that should highlight together (all points from one country, all bars in one facet); the SVG event system propagates by `data_id` automatically.
- Inside Shiny, swap `plotOutput()` and `renderPlot()` for `girafeOutput()` and `renderGirafe()`; selection and hover events become reactive inputs without additional plumbing.
- Customise hover and selection CSS with `opts_hover(css = ...)` and `opts_selection(css = ...)`; the same plot can present very different interactive feedback depending on context.
- Avoid `ggiraph` for plots with many tens of thousands of elements; SVG renders every element as a DOM node, and browsers slow down beyond a few thousand. For very large data, hexbin or canvas-rendered alternatives (`scattermore`, `plotly` with `scattergl`) scale better.
- For static publication, the same `ggplot` object renders without modification through `ggsave()` — write your interactive code over a normal `ggplot` and you get both versions for free.

R Packages Used

`ggiraph` for the interactive geoms, `girafe()`, and the `opts_*` helpers; `ggplot2` for the underlying grammar; `htmlwidgets` for embedding standalone widgets; `shiny` when reactive selection events drive downstream computation.

For Reviewers

What to look for in a paper using this method.

- **Common misapplications.**
- **Diagnostics that should be reported but often aren't.**
- **Red flags in tables and figures.**
- **What to verify.**
- **What an adequate Methods paragraph must contain.**

See also — labs in this chapter

- ggplot2 grammar and multi-panel layouts